Singleton

Building the intuition

Before we write a single line of code, let's understand *why* this pattern exists. Patterns are invented to solve real, recurring pain — Singleton is no different.

> **The analogy:** Think of a country's central bank. Every commercial bank, every government department, every trader interacts with exactly *one* central bank. You cannot spin up a second one — that would cause monetary chaos. The Singleton pattern is that central bank of your application's code.

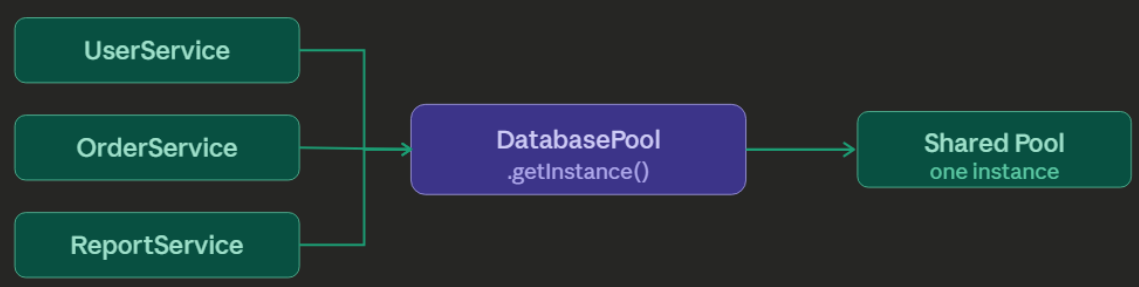
THE PROBLEM — WITHOUT SINGLETON

Imagine three services in your backend all needing a database connection. Without Singleton, each creates its own:

3 independent connections — memory waste, connection limit exhausted, no shared state

THE SOLUTION — WITH SINGLETON

All three services ask a single gatekeeper method for the shared pool. Only one instance ever lives in memory, no matter how many callers ask:



All callers share one instance — memory-efficient, consistent state, controlled access

The three components

Every Singleton in any language — Java, Python, Go, TypeScript — is built from exactly three pieces. Understand these and you can read or write any Singleton you encounter.

private

Private Constructor

Prevents anyone from calling `new DatabasePool()` from outside the class. Only the class itself can instantiate itself. This is the *lock* that enforces uniqueness — without it, any caller could create a second instance.

private static

Static Instance Field

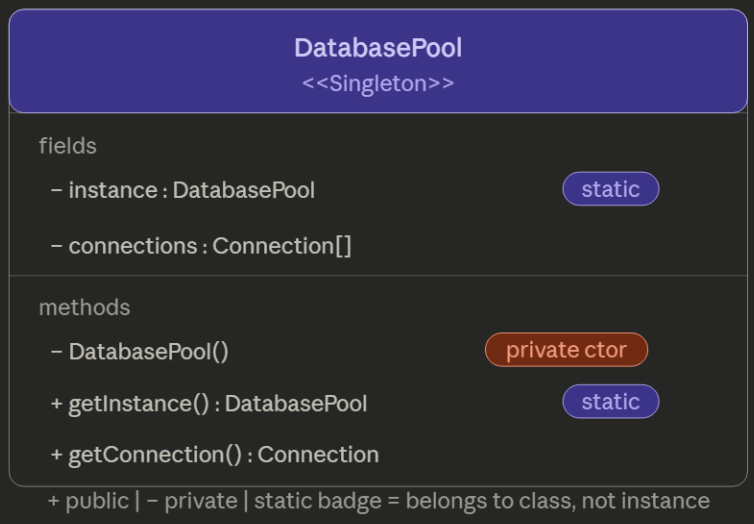
A class-level variable (not instance-level) that holds the one and only instance. Because it's `static`, it belongs to the *class itself* — not to any object. It's initialized to `null` and set exactly once.

public static

getInstance() Method

The only entrance into the class. Every caller goes through this method. On first call it creates the instance; on every subsequent call it returns the same one. This is the *gatekeeper*.

CLASS DIAGRAM



+ public | – private | static badge = belongs to class, not instance

Implementation flow

There are four common ways to implement Singleton in Java. Each one solves a gap left by the previous. I'll take you from the naive broken version to the production-ready one — so you understand *why* each step evolved, not just what the code looks like.

1 · Naive (broken) | 2 · Eager init | 3 · Double-checked lock | 4 · Enum (best)

1. Naïve

> Not thread-safe. Never ship this to production. It's here so you understand exactly what problem thread safety solves.

```java
public class DatabasePool {

    // Starts as null — no instance yet
    private static DatabasePool instance;

    // Private constructor — no one outside can call new DatabasePool()
    private DatabasePool() {
        // expensive setup: open connections, configure pool...
    }

    public static DatabasePool getInstance() {
        if (instance == null) {                // Thread A AND Thread B both see null
            instance = new DatabasePool();  // Both create an instance! Race condition.
        }
        return instance;
    }
}
```

> The race condition: Thread A checks `instance == null` → true. Before it finishes creating the instance, Thread B checks → also true. Now two threads each create their own instance. Your "singleton" has two copies in memory. Connections are doubled, config diverges.

2. Eager Init

```java
public class DatabasePool {

    // Instance created when the class is loaded by the JVM
    // Class loading is synchronized by the JVM — always thread-safe
    private static final DatabasePool INSTANCE = new DatabasePool();

    private DatabasePool() { }

    // No null check needed — INSTANCE is always ready
    public static DatabasePool getInstance() {
        return INSTANCE;
    }
}
```

> Thread-safe because the JVM guarantees class loading happens exactly once, atomically. The downside: the instance is created at app startup whether you ever use it or not. If initialization is expensive and the singleton might not be needed, this wastes resources. If it's always needed and cheap to init — use this, it's clean and simple.

3. Double-Checked Lock

```java
public class DatabasePool {

    // volatile: ensures every thread sees the latest write immediately
    // without it, CPU instruction reordering can break this pattern
    private static volatile DatabasePool instance;

    private DatabasePool() { }

    public static DatabasePool getInstance() {
        if (instance == null) {                    // Check 1 — fast path, no lock
            synchronized (DatabasePool.class) {  // Lock acquired by one thread
                if (instance == null) {          // Check 2 — now safely inside lock
                    instance = new DatabasePool();
                }
            }
        }
        return instance;                           // 99.9% of calls skip both checks
    }
}
```

> This is the recommended hand-written singleton for lazy initialization in Java. The first null check avoids locking on every call after initialization (fast path). The second null check inside the `synchronized` block stops the race condition. The `volatile` keyword prevents the CPU from reordering the write to `instance` before the constructor finishes.

4. Enum

```java
// Joshua Bloch's "Effective Java" recommendation — cleanest Java Singleton
// Thread-safe, serialization-safe, and reflection-attack-safe. All for free.
public enum DatabasePool {
    INSTANCE;  // JVM guarantees this is created exactly once

    private final List<Connection> connections = new ArrayList<>();

    // Your actual business methods go here
    public Connection getConnection() {
        return connections.get(0);
    }
}

// Usage — anywhere in your codebase:
Connection conn = DatabasePool.INSTANCE.getConnection();
```

> Enum constants are guaranteed by the JVM spec to be instantiated exactly once per class loader, and that instantiation is thread-safe. Enums also cannot be deserialized into new instances and cannot be bypassed via reflection. In practice, if you use Spring Boot, you won't write this either — Spring beans are singletons by default. But this is what you reach for in pure Java, library code, or an interview.

WHICH APPROACH FOR WHICH SITUATION?

Use Eager init when

The resource is always needed and cheap to initialize. You want the simplest possible implementation. Startup time is not a concern.

Use Double-checked lock when

Initialization is expensive and the singleton may not always be needed (lazy). You're in a multi-threaded environment — which, in any real backend, is always true.

Use Enum when

You want the most robust, idiomatic Java singleton. You need protection against serialization edge cases. You're not using a DI framework.

Use @Bean / @Component when

You're in Spring Boot. Annotate with `@Component`, `@Service`, or `@Bean`. Spring's default scope is singleton — it does all of this for you.

Use cases in backend development

These are the real places you'll encounter or apply Singleton as a backend developer. The common thread across all of them: *expensive to create, safe to share, needed globally*.

Most common

Database connection pool

Opening a DB connection is expensive — TCP handshake, authentication, SSL negotiation. A pool pre-creates N connections and reuses them. One pool shared across the entire app. HikariCP (used by Spring Boot by default) is always a singleton. Never create a new pool per request.

Most common

Application config manager

Your app reads `.env`, `application.yml`, or environment variables exactly once at startup. Every service needing a DB URL, API key, feature flag, or timeout value queries the same config object. Spring's `Environment` bean is a singleton. Consistency is guaranteed.

Logger

A logging framework (SLF4J, Log4j, Logback) maintains one handler per logger name. If two loggers existed for the same class, you'd get duplicate log entries and file-locking conflicts. `LoggerFactory.getLogger(MyClass.class)` always returns the same logger for the same class name.

Cache client (Redis / Memcached)

Your Redis client holds a connection pool to the Redis server. Creating a new Jedis or Lettuce client per request would exhaust server-side connections within seconds under any real load. One client singleton shared across all request handlers. Declare as a Spring `@Bean` and inject everywhere.

Thread pool / Executor service

A backend that calls `new Thread()` for every incoming request will crash under any meaningful load — thread creation has real cost, and unbounded threads exhaust JVM heap. Instead, you create one `ExecutorService` at startup and submit tasks to it. In Java, your `Executors.newFixedThreadPool(n)` lives as a singleton. Spring's `@Async` and `@Scheduled` manage their thread pools the same way behind the scenes.

> Notice the pattern in all five: the resource is *expensive to create* (cost is paid once), *safe to share* (internal thread safety handled), and *needed from many places* (global access is justified). If your use case doesn't check all three boxes, Singleton is probably the wrong tool.

Pros & cons

Singleton is one of the most used — and most misused — patterns. What separates a junior from a senior is knowing when it helps vs. when it quietly makes your codebase worse.

Pros

- Controlled access — one instance means no duplicates, no diverging state
- Memory efficient — expensive resources allocated once, reused forever
- Global access without threading references through every method call
- Lazy initialization — the instance is only created when first requested
- Consistent view — all callers see and modify the same object

Cons

- Hard to unit test — global singletons make isolation and mocking painful
- Hidden global state — a bug in the singleton silently corrupts the entire app
- Thread safety must be implemented correctly — naive singleton breaks under concurrency
- Violates Single Responsibility — the class manages both its own lifecycle and its logic
- Breaks in distributed systems — each JVM/pod gets its own "singleton"; they drift

THE SENIOR ENGINEER'S PERSPECTIVE

> In modern backend development with Spring Boot, you rarely write a Singleton by hand. Every `@Component`, `@Service`, `@Repository`, and `@Bean` is a singleton by default — Spring creates it once and injects the same instance everywhere via dependency injection. Understanding the Singleton *pattern* is what lets you reason about Spring's behavior, debug lifecycle issues, and make smart scope decisions (`singleton` vs `prototype` vs `request`).